This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

On the Role of Temporal Granularity in the Robustness of Spiking Neural Networks

Mengting Xu[1,2], Shi Gu[1,2], Peng Lin[1,2], De Ma[1,2], Huajin Tang[1,2], Qian Zheng[1,2]*, Gang Pan[1,2] *
[1]The State Key Lab of Brain-Machine Intelligence, Zhejiang University
[2]College of Computer Science and Technology, Zhejiang University
{xumengting, gus, penglin, made, htang, qianzheng, gpan}@zju.edu.cn

Abstract

As the third generation of neural networks, Spiking Neural Networks (SNNs) have demonstrated remarkable potential across diverse applications owing to their unique temporal dynamics. In recent years, analyzing the robustness of SNNs from a temporal perspective has become an emerging research focus. However, most existing works examine only the overall temporal behavior of SNNs, typically applying adversarial attacks that rely on time-averaged gradients. In this study, we revisit SNN robustness through the lens of temporal granularity, emphasizing the distinct behaviors that occur at individual time steps. We first introduce a Temporal Granularity Attack (TG-Attack), which selectively perturbs gradients at specific time steps. This approach enables a finer-grained evaluation of SNN robustness across time and demonstrates higher attack success rates than traditional gradient-averaging methods. Furthermore, we theoretically show that the robustness of SNNs at a given time step is determined by the Hessian of the input–output gradient at that step, which we define as Temporal Sensitivity (TS). By calculating the Temporal Sensitivity Value (TSV) for each time step, robustness can be effectively estimated without generating adversarial examples. Finally, we propose a Temporal Granularity Regularization (TG-Reg) term that constrains the TSV across all time steps, thereby improving the model's overall robustness. Experimental evaluations confirm that our framework consistently outperforms existing state-of-the-art methods. Code is available at https://github.com/zju-bmi-lab/TG-SNN-code.

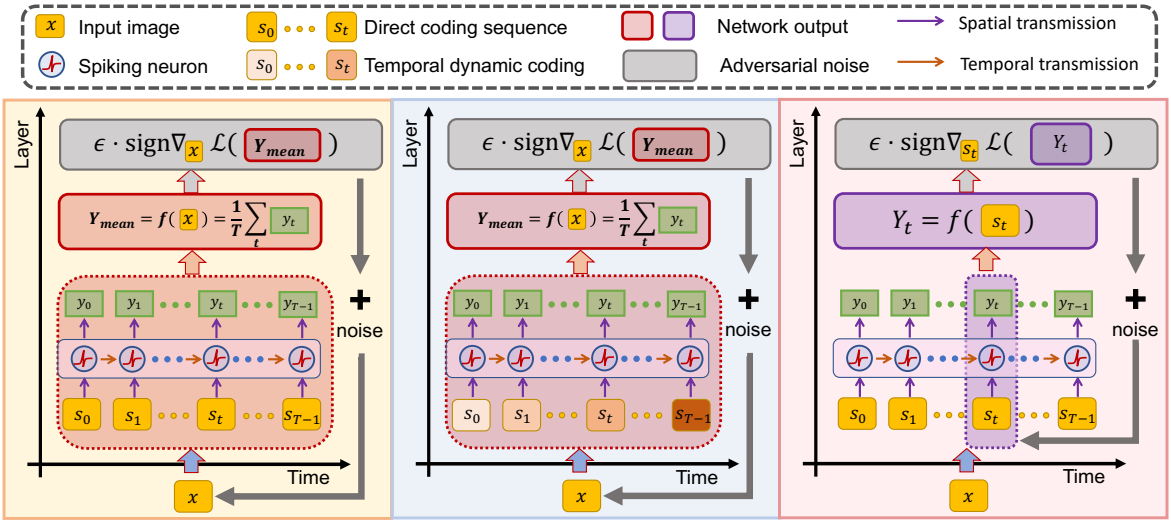
1. Introduction

Recent studies have demonstrated that the intelligence of organisms is closely linked to their temporal processing mechanisms [13, 39, 41, 42]. As the third generation of neural networks, Spiking Neural Networks (SNNs) are widely utilized in various tasks due to their inherent temporal characteristics, such as spatio-temporal pattern recognition [4, 15, 26, 37], efficient temporal image classification [7, 33], and temporal spiking early-exit neural networks [23], among others. Additionally, investigating the robustness of SNNs through temporal characteristics has increasingly garnered attention from researchers.

Among various implementation strategies for SNNs, surrogate gradient-based direct training methods have emerged as the mainstream approach [38]. Nevertheless, recent findings suggest that SNNs with direct encoding method tend to be less robust than their rate-encoded counterparts [32, 36]. These observations highlight the need for a deeper investigation into the robustness of direct trained SNNs with direct encoding. Therefore, this work specifically focuses on the robustness of SNNs under direct encoding and surrogate gradient-based direct training method. From the perspective of temporal characteristics, current approaches to studying the robustness of SNNs can be broadly divided into two categories: The first category investigates robustness by combining the temporal and spatial characteristics of SNNs, as shown in Figure 1(a). It includes investigating how unique SNN parameters (*e.g.*, leak factors [35], thresholds [12], encoding [30]) influence robustness. It also encompasses improved training strategies like parameter regularization (*e.g.*, RAT [8]), adversarial training (*e.g.*, AT [20]), and stochastic gating regularization [11]. The second category leverages the temporal dynamics of SNNs (*e.g.*, spike timing coding [9], temporal frequency coding [40], and random smoothing coding [36]) to analyze robustness, as depicted in Figure 1(b). Currently, these methods primarily concentrate on the overall temporal characteristics of SNNs, with robustness test conducted by constructing adversarial attacks based on the *average gradient* over time, as illustrated in the attack strategies shown in Figure 1(a)(b).

However, existing research indicates that the temporal learning ability of SNNs varies across time steps [7, 33], and the gradients of SNNs differ at each time step [18, 37].

*Corresponding author.



(a) Attack based on average gradient (Combine temporal and spatial characteristics)

(b) Attack based on average gradient (Temporal dynamic coding)

(c) Attack based on temporal granularity **(Ours)**

Figure 1. Strategies used by attack methods to handle temporal characteristics. (a) Perturbation-generation pipeline based on **direct encoding** and the **average temporal gradient**. (b) Perturbation-generation pipeline based on **temporal encoding** and the **average temporal gradient**. (c) TG-attack: perturbations constructed **from the temporal gradient at each time step**, applicable with either **direct encoding** or the **temporal encoding**.

Therefore, it is also crucial to investigate the impact of gradients at temporal granularity (*i.e.*, at individual time steps) on the robustness of SNNs. In this paper, first, different from traditional adversarial attacks based on the *average gradient* of temporal characteristics [8, 11, 35], we generate adversarial attacks targeting the *gradients at temporal granularity* of SNNs (as shown in Figure 1(c)). Our findings reveal that robustness of SNNs differs across time steps, and adversarial attacks generated at each time step are more effective than those generated based on average gradients. Then, building on this experimental observation, we theoretically analyze the robustness of SNNs at temporal granularity. We derive that robustness at a specific time step is constrained by Hessian matrix of input-output gradient at that time step, which we defined as Temporal Sensitivity (TS). Furthermore, based on this theoretical analysis, we introduce a temporal granularity robustness regularization term (TG-Reg) that constrains the Temporal Sensitivity Value (TSV) at each time step, thereby effectively enhancing overall robustness of SNNs. Our main contributions are:

- We propose a temporal granularity attack (TG-attack) targeting the gradients of SNNs at individual time steps. This method not only reflects the robustness of SNNs across different time steps but also demonstrates superior attack efficacy compared to traditional approaches.
- We provide a theoretical analysis showing that the robustness of SNNs at temporal granularity is constrained by the Hessian matrix of the input-output gradients at that specific time step, which we refer to as Temporal Sensitivity (TS). By calculating the Temporal Sensitivity Values (TSV) at different time steps, we can assess the robustness of SNNs without the need to generate adversarial examples for robustness test.
- We propose a temporal granularity robustness regularization term (TG-Reg) that enhances the robustness of SNNs by constraining their TSV at each time step. Experimental results demonstrate that our approach outperforms state-of-the-art methods.

2. Related Work

From the perspective of temporal characteristics, current approaches to studying the robustness of SNNs can be broadly categorized into two groups.

Combining the temporal and spatial characteristics. The first group includes investigations into how specific SNN parameters [28, 31] influence robustness. For instance, El-Allami *et al.* demonstrate that the inherent robustness of SNNs is highly dependent on time windows and firing voltage thresholds [12]. Sharmin *et al.* further confirm that the leak factor provides an additional mechanism to control adversarial perturbations [32]. The StoG model [11] introduces stochasticity to effectively enhance robustness. Additionally, this group involves the development of improved training strategies. Kundu *et al.* introduce HIRE-SNN to improve the efficiency of adversarial training in SNNs [20]. Ding *et al.* propose SNN-RAT, which applies a regularized adversarial training scheme with low computational overhead [8]. Liu *et al.* suggest minimizing the

performance gap between an SNN subjected to adversarial perturbations and random perturbations, thereby improving its overall robustness [25].

Temporal dynamic based on different coding. The second group focuses on leveraging the temporal dynamics of SNNs. Recent work systematically explores the timing mechanisms of spike coding in SNNs [9], suggesting that spike timing coding can enhance robustness against attacks. FEEL-SNN [40] leverages the temporal dynamic characteristics of SNNs by designing a novel coding method that enables SNNs to learn frequency information at different time steps, significantly improving robustness. RSC-SNN proposes randomized smoothing coding method which adds noise on every time step to improve model robustness [36].

However, existing methods explore the relationship between model robustness and temporal characteristics typically focus on overall temporal characteristics of SNNs. In contrast, this work approaches the analysis of SNN robustness from the perspective of temporal granularity. More related work of adversarial attacks are presented in Appendix.

3. Preliminaries

3.1. Spike Neuron Dynamics in SNNs

Leaky Integrate-and-Fire (LIF) neurons [7, 34, 35] are extensively employed in SNNs to emulate the information transmission mechanisms of the human brain. LIF neurons convey information through spikes, with their dynamics divided into three stages: membrane potential accumulation, spike generation, and membrane potential reset. These dynamics are described by the following equations:

$$\mathbf{u}_t^l = \tau(\mathbf{u}_{t-1}^l - V_{th}\mathbf{s}_{t-1}^l) + \mathbf{W}^l\mathbf{s}_{t-1}^l, \tag{1}$$

$$\mathbf{s}_t^l = H(\mathbf{u}_t^l - V_{th}), \tag{2}$$

where $\mathbf{u}_t^l$ and $\mathbf{s}_t^l$ represent the membrane potential and output spike of neurons in layer l at time t, respectively. $\mathbf{W}^l$ denotes the linear connections between layers $l-1$ and l, τ is membrane potential leak factor, and V_{th} is threshold. Equation (1) describes both the accumulation of membrane potential and the reset process, where the membrane potential is reset to zero after firing, for simplicity. Equation (2) describes the spike generation process, where a spike $\mathbf{s}_t^l$ is produced if the membrane potential $\mathbf{u}_t^l$ exceeds the threshold V_{th}, as determined by the Heaviside step function $H(\cdot)$.

3.2. Gradient-based Adversarial Attack

Consider a classification model f with dataset $(\boldsymbol{x}, y_{true})$, where $\boldsymbol{x}$ is the clean image and y_{true} is the corresponding label. The goal of an adversarial attack is to generate an adversarial example $\hat{\boldsymbol{x}}$ that satisfies:

$$\arg\max_y f(y|\hat{\boldsymbol{x}}) \neq \arg\max_y f(y|\boldsymbol{x}) \quad s.t. \quad ||\hat{\boldsymbol{x}} - \boldsymbol{x}||_p \leq \epsilon, \tag{3}$$

where $f(\cdot)$ is output probability, $||\cdot||_p$ denotes the L_p-norm. In this work, we utilize L_∞-norm, and ϵ constrains the perturbation strength.

Gradient-based adversarial attack methods are currently the most prevalent and effective. These methods perturb the original data $\boldsymbol{x}$ along the direction of the gradient sign of the loss function, either in one or multiple steps, to increase the perturbed linear output and deceive the network. Notable gradient-based attack methods include FGSM [14], PGD [27], and BIM [21]. For instance, the one-step adversarial attack FGSM is defined as:

$$\hat{\boldsymbol{x}} = \boldsymbol{x} + \epsilon \text{sign}(\nabla_{\boldsymbol{x}}\mathcal{L}(f(\boldsymbol{x}), y_{true})), \tag{4}$$

where $\text{sign}(\cdot)$ is a function that extracts the sign of a real number. $\mathcal{L}(\cdot)$ is the loss function.

In SNNs, the non-differentiability of spike firing (Equation (2)) necessitates the use of surrogate gradients [7], enabling the successful implementation of gradient-based attack methods. To launch an attack on images, as illustrated in Figure 1(a), the first step is to encode the images into sequences $\varsigma(\boldsymbol{x}) = \boldsymbol{s} =< \boldsymbol{s}_0, \boldsymbol{s}_1, \ldots, \boldsymbol{s}_{T-1} >$, where $\varsigma : \mathbb{R}^{C\times H\times W} \rightarrow \mathbb{R}^{T\times C\times H\times W}$, and T denotes the number of timesteps. Here, C, H, W represent the number of channel, height, and width, respectively. The most common encoding method involves replicating $\boldsymbol{x}$ across the temporal dimension [17]. Referring to the gradient $\nabla_{\boldsymbol{x}}\mathcal{L}(f(\boldsymbol{x}), y_{true})$ in Equation (4), the gradient with respect to $\boldsymbol{x}$ can be computed from the gradient with respect to $\boldsymbol{s}$ as follows:

$$\nabla_{\boldsymbol{x}}\mathcal{L}(f(\boldsymbol{x}), y_{true}) = \frac{\partial \mathcal{L}}{\partial \boldsymbol{x}} := \frac{1}{T}\sum_t \frac{\partial \mathcal{L}}{\partial \boldsymbol{s}_t}. \tag{5}$$

4. Methods

4.1. Temporal Granularity Adversarial Attack

Let $\mathcal{X} \subseteq \mathbb{R}^{C\times H\times W}$ denote input space, let $\mathcal{Y} = \{1, \ldots, K\}$ be a finite set of K possible class labels. A classifier $f_{\boldsymbol{w}}$ with parameter $\boldsymbol{w}$ predicts the probability of an input sample $\boldsymbol{x} \in \mathcal{X}$ belonging to class label $y \in \mathcal{Y}$, expressed as

$$f_{\boldsymbol{w}}(y|x) = p_y(\boldsymbol{x}, \boldsymbol{w}) = \exp(z_y(\boldsymbol{x}, \boldsymbol{w}))/\sum_{y'=1}^{K}\exp(z_{y'}(\boldsymbol{x}, \boldsymbol{w})), \tag{6}$$

where $z_y(\boldsymbol{x}, \boldsymbol{w})$ is the logit output corresponding to class y, and $p_y(\boldsymbol{x}, \boldsymbol{w})$ represents the probability that $\boldsymbol{x}$ belonging to class y. For simplicity, we use f to denote $f_{\boldsymbol{w}}$ in our paper.

Analysis of traditional attacks based on an average gradient at the temporal characteristic. By combining Equations (4) and (5), we observe that this method generates the perturbation based on the average gradient over the time (*i.e.*, $\frac{1}{T}\sum_t \frac{\partial \mathcal{L}}{\partial \boldsymbol{s}_t}$), which is then added to the original,

unencoded image $\boldsymbol{x}$, as shown in Figure 1(a). This approach suppresses directions that are strong at particular time steps and neglects the impact of gradient variations at individual timesteps on the robustness of SNNs, which may lead the current attack methods are insufficiently effective, and the defenses relying on these methods may have limited capacity to defend against strong attacks.

In this work, we focus on constructing perturbations based on the gradients at individual timesteps of the SNN to further analyze its robustness at temporal granularity. By leveraging the temporal characteristics of SNNs, we propose the gradient-based temporal granularity adversarial attack (TG-attack) (as shown in Figure 1(c)). Using a one-step attack as an example, the formulation of our TG-attack is as follows:

$$\hat{\boldsymbol{s}_t} = \boldsymbol{s}_t + \epsilon \text{sign}(\nabla_{\boldsymbol{s}_t} \mathcal{L}(f(\boldsymbol{s}_t), y_{true})), \tag{7}$$

$$\nabla_{\boldsymbol{s}_t} \mathcal{L}(f(\boldsymbol{s}_t), y_{true}) = \frac{\partial \mathcal{L}[t]}{\partial \boldsymbol{s}_t}. \tag{8}$$

$$\hat{\boldsymbol{x}} = < \hat{\boldsymbol{s}_0}, \hat{\boldsymbol{s}_1}, \ldots, \hat{\boldsymbol{s}_{T-1}} > . \tag{9}$$

The innovation of the TG-attack lies in two key aspects. First, we construct perturbations for the current input ($\hat{\boldsymbol{s}_t}$) at each time step (Equation (7)) based on the gradient at that specific time step (Equation (8)). Notably, since the output distribution of SNNs is influenced by information from all previous timesteps with LIF neurons, we compute the loss function at the current time step ($\mathcal{L}[t]$) using the output ($f(\boldsymbol{s}_t)$), where $f(\boldsymbol{s}_t)$ is computed by accumulating the spikes at the output layer across t steps. Second, the final adversarial example ($\hat{\boldsymbol{x}}$) is constructed by combining the temporal granularity adversarial examples ($\hat{\boldsymbol{s}_t}$) over time. TG-attack instead selects the best direction for each time step, preserving those time-specific directions and producing more effective perturbations.

In summary, our TG-attack not only reflects the robustness of SNNs at each timestep but also demonstrates superior attack effectiveness compared to traditional approaches, as shown in the experimental section.

4.2. Temporal Sensitivity Value Derivation

As with the TG-attack proposed above, a natural approach to enhancing the robustness of SNNs is to improve their robustness at each time step. In this section, we theoretically derive the Temporal Sensitivity Values (TSV) at each time step to quantify robustness at a temporal granularity. According to Equation (6), $f_{\boldsymbol{w}}(y|x)$ represents the full predictive distributions over all classes (i.e., softmax outputs), when a small perturbation ϵ is added to the input $\boldsymbol{s}_t$ at a single time step t, the robustness of SNNs at that time step can be measured using the Kullback-Leibler (KL) divergence [16], denoted as $\text{KL}(f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)|f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t+\epsilon))$ [18, 29]. Then, we have:

$$\begin{aligned} &\text{KL}(f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)||f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t+\epsilon)) = \\ &\mathbb{E}_{f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)}[\log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t) - \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t+\epsilon)]. \end{aligned} \tag{10}$$

Using a second-order Taylor expansion [24], $\log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t+\epsilon)$ can be approximated as:

$$\begin{aligned} &\log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t+\epsilon) \approx \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t) + \\ &\epsilon^\top \nabla_{\boldsymbol{s}_t} \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t) + \frac{1}{2}\epsilon^\top H_{\boldsymbol{s}_t} \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)\epsilon, \end{aligned} \tag{11}$$

yielding:

$$\begin{aligned} &\text{KL}(f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)\|f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t+\epsilon)) \approx \\ &\mathbb{E}_{f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)}[-\epsilon^\top \nabla_{\boldsymbol{s}_t} \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t) \\ &- \frac{1}{2}\epsilon^\top H_{\boldsymbol{s}_t} \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)\epsilon], \end{aligned} \tag{12}$$

where $H_{\boldsymbol{s}_t} \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)$ represents the Hessian matrix of $\log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)$ with respect to temporal input $\boldsymbol{s}_t$.

By combining Equations (10), (11), and (12), we derive a constraint on the robustness of SNNs at time t, formalized in Theorem 1 (a detailed proof is provided in Appendix).

Theorem 1 *Given the robustness of SNNs at time t as $KL(f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)||f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t+\epsilon))$, where ϵ is the perturbation, $f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)$ is the temporal output probability at time t, $\boldsymbol{w}$ is the model parameters. We have*

$$\begin{aligned} &KL(f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)||f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t+\epsilon)) \approx \\ &\frac{1}{2}\epsilon^\top \mathbb{E}_{f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)}[-H_{\boldsymbol{s}_t} \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)]\epsilon, \end{aligned} \tag{13}$$

where $H_{\boldsymbol{s}_t} \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)$ represents the Hessian matrix of $\nabla_{\boldsymbol{s}_t} \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)$ with respect to temporal input $\boldsymbol{s}_t$.

Theorem 1 reveals that the robustness of SNNs at a single time step is governed by the Hessian matrix of input-output gradients, which we refer to as Temporal Sensitivity Matrix (TSM). The Temporal Sensitivity Matrix (TSM) and temporal sensitivity value (TSV) are defined as:

$$TSM(\boldsymbol{s}_t) = \mathbb{E}_{f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)}[-H_{\boldsymbol{s}_t} \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)]. \tag{14}$$

$$TSV(\boldsymbol{s}_t) = \sum_{i,j} \mathbb{E}_{f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)}[-H_{\boldsymbol{s}_t} \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)]_{i,j}, \tag{15}$$

where i, j are index of temporal sensitivity matrix.

In summary, we provide a theoretical analysis demonstrating that the robustness of SNNs at temporal granularity is constrained by the Hessian matrix of the input-output gradients at each specific time step (Equation (14)), which we call Temporal Sensitivity Matrix (TSM). By computing the Temporal Sensitivity Values (TSV) (Equation (15)) at different time steps, we can evaluate the robustness of SNNs without requiring the generation of adversarial examples for robustness test.

4.3. Temporal Granularity Robust Regularization

The theoretical analysis for SNNs robustness from temporal granularity above provides us the motivation for exploiting temporal sensitivity value (TSV) as a robustness regularization term for SNN robustness enhancement.

Based on the robustness analysis of SNNs at the individual time step, we can conclude that a higher Temporal Sensitivity Value (TSV) indicates that the SNN is more susceptible to perturbations at that specific time step, for its KL divergence becomes larger. Therefore, to enhance the overall robustness of SNNs, we focus on minimizing the temporal sensitivity value at each time step in our proposed regularization. The proposed Temporal Granularity regularization (TG-Reg) is presented as follows.

$$\mathcal{L}_{TG} = \sum_{t=0}^{T} TSV(\boldsymbol{s}_t). \tag{16}$$

And the overall loss function can be described as:

$$\mathcal{L}(f(\boldsymbol{x}), y_{true}) = \mathcal{L}_{CE}(f(\boldsymbol{x}), y_{true}) + \lambda \mathcal{L}_{TG}, \tag{17}$$

where $\mathcal{L}_{CE}(f(\boldsymbol{x}), y_{true})$ represents the commonly used cross-entropy loss. λ is a tunable scaling parameter which is fixed for TSV.

However, calculating the second-order Hessian matrix (Equation (15)) during training is time-consuming, we now optimize the form of temporal sensitivity value at each time step as follows:

Expanding the temporal sensitivity value $TSV(\boldsymbol{s}_t)$:

$$\begin{aligned} \nabla_{\boldsymbol{s}_t} \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t) &= \frac{\nabla_{\boldsymbol{s}_t} f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)}{f_w(y_t|\boldsymbol{s}_t)}, \\ H_{\boldsymbol{s}_t} \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t) &= \nabla_{\boldsymbol{s}_t}(\frac{\nabla_{\boldsymbol{s}_t} f_w(y_t|\boldsymbol{s}_t)}{f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)}). \end{aligned} \tag{18}$$

Thus,

$$\begin{aligned} &H_{\boldsymbol{s}_t} \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t) = \\ &\frac{H_{\boldsymbol{s}_t} f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)}{f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)} - \frac{\nabla_{\boldsymbol{s}_t} f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t) \nabla_{\boldsymbol{s}_t} f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)^\top}{f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)^2}. \end{aligned} \tag{19}$$

Thus, the temporal sensitivity value $TSV(\boldsymbol{s}_t)$ becomes:

$$\begin{aligned} TSV(\boldsymbol{s}_t) = \sum_{i,j} \mathbb{E}_{f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)}[&\frac{\nabla_{\boldsymbol{s}_t} f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t) \nabla_{\boldsymbol{s}_t} f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)^\top}{f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)^2} \\ &- \frac{H_{\boldsymbol{s}_t} f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)}{f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)}]_{i,j}. \end{aligned} \tag{20}$$

By combining Equation (15) - Equation (20), the final form of the temporal sensitivity value $TSV(\boldsymbol{s}_t)$ is (the detailed proof can be found in Appendix):

$$\begin{aligned} TSV(\boldsymbol{s}_t) \approx \sum_{i,j} \mathbb{E}_{f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)}[\\ \nabla_{\boldsymbol{s}_t} \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t) \nabla_{\boldsymbol{s}_t} \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)^\top]_{i,j}, \end{aligned} \tag{21}$$

where $\nabla_{\boldsymbol{s}_t} \log f_{\boldsymbol{w}}(y_t|\boldsymbol{s}_t)$ is the first-order gradient of input and output at time step t. Combining Equations (21), (16), and (17), the proposed temporal granularity regularization term can be seamlessly integrated into the model training process to improve robustness.

Differences with previous work SR [25]. SR enhances the robustness of SNNs through gradient sparsity regularization. Our work differs from SR in three key aspects: (1) Gradient Computation: SR focuses on output probability of the true label for input image, computing the gradient of the most probable class *averaged over time steps*. In contrast, TG-Reg computes the gradient magnitude *independently at each time step*. (2) Perturbation Target: SR calculates gradients with respect to the original input image x whereas we compute gradients with respect to the encoded input s_t at each time step (see Equation (8)(9)). (3) Regularization Implementation: SR integrates a finite difference approximation into the original loss function. In contrast, TG-Reg imposes a regularization constraint based on the input-output gradient at each time step (see Equation (21)).

5. Experiments

5.1. Experimental Setup

The datasets we used are CIFAR-10, DVS-CIFAR10 [22], CIFAR-100 [19], and Tiny-ImageNet [6]. The networks include SNN versions of VGG11, WideResNet16, ResNet19, and ResNet34. We use the triangle-like surrogate functions [7]. The adversarial training strategy we used for comparison uses examples from PGD attacks ($\epsilon = 2/255$, iterative step $k = 2$). The attack methods include FGSM [14], PGD with random start [27], BIM [21], CW [3], AutoAttack [5], EOT [2] and BPDA [1]. Unless otherwise stated, for all attacks, we set perturbation $\epsilon = 8/255$, iterative step $k = 7$ and step size $\alpha = 0.01$. The time step to infer SNN is set to 8 by default. We set $\lambda = 100$ in Equation (17). The sensitivity of parameter λ can be found in Appendix. In our work, the training process lasts for 200 epochs for all experiments. Batch normalization are used to overcome the gradient vanishing or explosion. SGD optimizer is deployed, and the initial learning rate is set to 0.1. The learning rate uses a cosine annealing schedule with T_{max} equaling the max number of epochs. All the experiments are conducted on the PyTorch platform on NVIDIA RTX 3090.

5.2. Effectiveness of Temporal Granularity Attack

TG-attack differs significantly from prior gradient-based attacks by utilizing the gradient at each time step rather than averaging it over time. Beyond the methodological distinction, we evaluate the effectiveness of our attack. Specifically, we apply TG-attack (*i.e.*, TG-FGSM, TG-PGD and TG-BIM) and compare its performance against traditional average gradient-based multi-step attacks (*i.e.*, FGSM, PGD

Table 1. Performance of the proposed TG-attack and TG-Reg with different settings. The results of TG-attack are shown by underline. The results with better robustness are highlighted in **bold**. The number in "Settings" represents time steps.

Train with original samples												
Settings	Methods	Clean	FGSM	TG-FGSM	PGD	TG-PGD	BIM	TG-BIM	CW	EOT	BPDA	AA
CIFAR10, VGG11, 4	Vanilla	92.42	12.71	9.72	0.38	0.21	0.23	0.18	10.17	0.38	70.35	0.10
CIFAR10, VGG11,4	TG-Reg (Ours)	90.89	**28.79**	**25.28**	**4.25**	**3.20**	**3.56**	**2.72**	**11.55**	**4.25**	**73.33**	**2.36**
CIFAR10, ResNet34, 4	Vanilla	93.17	50.61	48.39	8.91	5.61	8.63	7.84	10.34	2.92	62.88	0.08
CIFAR10, ResNet34,4	TG-Reg (Ours)	87.32	**51.52**	**50.35**	**18.46**	**14.05**	**17.37**	**14.76**	**11.09**	**18.70**	**71.09**	**1.23**
CIFAR100,VGG11,8	Vanilla	72.76	4.61	4.99	0.12	0.07	0.10	0.03	1.11	0.11	46.28	0.07
CIFAR100,VGG11,8	StoG [11]	70.44	8.27	-	0.49	-	-	-	-	-	-	-
CIFAR100,VGG11,8	DLIF [10]	70.79	6.95	-	0.08	-	-	-	-	-	-	-
CIFAR100,VGG11,8	FEEL [40]	73.79	9.60	-	2.04	-	1.81	-	-	-	-	-
CIFAR100,VGG11,8	TG-Reg (Ours)	70.98	**12.99**	**11.37**	**4.59**	**3.99**	**4.09**	**3.55**	**2.24**	**4.57**	**48.63**	**1.89**
CIFAR100,WR16,8	Vanilla	74.51	6.69	4.36	0.02	0.01	0.03	0.01	8.29	0.01	54.83	1.09
CIFAR100,WR16,8	TG-Reg (Ours)	72.78	**8.20**	**5.89**	**0.33**	**0.18**	**0.29**	**0.11**	**10.34**	**4.62**	**58.34**	**2.21**
T-ImageNet,ResNet19,4	Vanilla	43.70	7.92	6.02	2.91	1.83	2.90	1.56	7.54	1.34	30.02	2.13
T-ImageNet,ResNet19,4	TG-Reg (Ours)	42.12	**8.64**	**7.45**	**3.87**	**2.89**	**4.03**	**2.78**	**9.08**	**3.45**	**33.23**	**3.70**
DVSCIFAR10, VGG11,10	Vanilla	72.70	17.90	13.60	11.20	6.50	11.00	6.80	12.09	2.56	49.12	1.45
DVSCIFAR10, VGG11,10	TG-Reg (Ours)	70.00	**36.20**	**35.30**	**31.70**	**29.70**	**32.10**	**29.70**	**13.10**	**5.21**	**52.09**	**2.34**
Train with adversarial examples												
Settings	Methods	Clean	FGSM	TG-FGSM	PGD	TG-PGD	BIM	TG-BIM	CW	EOT	BPDA	AA
CIFAR10, VGG11, 4	AT [27]	90.56	37.88	33.87	20.01	17.11	18.88	15.77	10.59	20.32	73.04	6.25
CIFAR10, VGG11, 4	RAT [8]	90.52	**40.78**	**37.65**	**22.35**	**19.24**	**20.88**	**17.66**	10.70	22.18	74.05	7.04
CIFAR10, VGG11, 4	SR [25]	90.96	36.87	33.32	13.11	12.14	11.43	10.49	10.43	13.28	71.11	6.33
CIFAR10, VGG11, 4	TG-Reg (Ours)	89.45	**48.64**	**44.48**	**31.12**	**27.54**	**29.40**	**25.61**	**10.71**	**30.74**	**74.41**	**8.76**
CIFAR100, VGG11, 8	AT [27]	66.72	17.14	15.87	9.76	8.60	9.10	8.02	1.21	9.87	47.52	0.89
CIFAR100, VGG11, 8	RAT [8]	68.72	19.07	18.91	10.42	9.58	9.73	8.71	1.25	10.40	47.53	2.32
CIFAR100, VGG11, 8	SR [25]	67.76	11.48	10.31	3.62	2.89	3.41	2.79	1.26	3.57	40.13	1.67
CIFAR100, VGG11, 8	TG-Reg (Ours)	68.06	**20.67**	**20.37**	**11.83**	**11.57**	**11.23**	**10.01**	**1.40**	**11.70**	**48.72**	**4.72**

and BIM) under various configurations.

As shown in Table 1, our TG-attack approach leads to greater accuracy degradation, demonstrating its higher effectiveness. For example, when trained with original samples, the accuracy of VGG11 drops from 12.71% to 9.72% attacked by TG-FGSM (CIFAR10 with $T = 4$, Vanilla). Similar trends are also observed when trained with adversarial examples, further validating the superior efficacy of our approach compared to conventional methods.

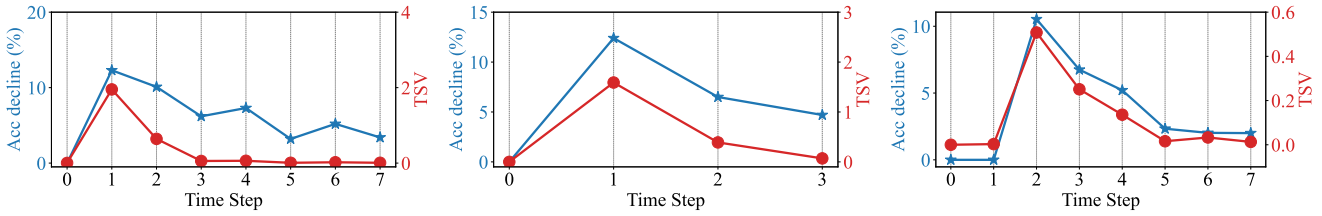
5.3. Effectiveness of Temporal Sensitivity Value

We reveal that the robustness of SNNs at each time step is constrained by TSV. Now, we validate the effectiveness of TSV through experiments. We record the changing of TSV and the accuracy decline of SNNs when attacked by FGSM at each time step. As shown in Figure 2, we find that the trend of TSV variations at each time step aligns with the decline in original accuracy observed under attack. Specifically, the larger the TSV at a given time step, the greater the robustness drop at that time step. This is consistent with our theoretical analysis. The experimental results validate the effectiveness of the proposed TSV and demonstrate that by calculating the TSV at different time steps, we can conveniently evaluate the robustness of SNNs at each step without the need to generate adversarial examples explicitly.

5.4. Overall Performance of Proposed TG-Reg

5.4.1. Train with original samples

First, we incorporate the proposed TG-Reg into the standard training framework for SNNs (*i.e.*, train with original samples). Experimental results are presented on various datasets (CIFAR-10, CIFAR-100 [19], Tiny-ImageNet [6] and DVS-CIFAR10 [22]) and network architectures (VGG11, WideResNet16, ResNet19, and



(a) CIFAR100,VGG11,$T = 8$ (b) CIFAR10,VGG11,$T = 4$ (c) CIFAR10,VGG11,$T = 8$

Figure 2. The changing of Temporal Sensitivity Value (TSV) of SNNs and the accuracy decline when attacked by FGSM at each time step.

Table 2. Performance of proposed TG-Reg under black-box attack settings. The datasets are CIFAR10 with VGG11 on $T = 4$ and CIFAR100 with VGG11 on $T = 8$.

Settings	Methods	FGSM	PGD	BIM	EOT	BPDA
CIFAR10	Vanilla	28.60	8.58	7.94	8.27	78.82
CIFAR10	**TG-Reg (Ours)**	**42.02**	**23.06**	**21.88**	**23.32**	**78.63**
CIFAR100	Vanilla	13.89	5.77	5.30	5.74	54.30
CIFAR100	**TG-Reg (Ours)**	**38.83**	**35.91**	**34.70**	**35.28**	**57.25**

ResNet34) with different time steps, under both traditional attacks and TG-attacks, as summarized in Table 1 . It can be observed that, across all attack methods, TG-Reg demonstrates consistent improvements in model robustness, as evidenced by the observed increases in accuracy. For instance, on VGG11 with CIFAR-10, TG-Reg enhances robustness by up to 16% and 4% under FGSM and PGD attacks when compared with vanilla method, respectively, at a time step of 4. Furthermore, we validate the effectiveness of our method by comparing it with current state-of-the-art methods, *i.e.*, StoG [11], DLIF [10], and FEEL-SNN [40], where the reported results are consistent with their original papers. It can be observed that, when compared with FEEL-SNN, TG-Reg also enhances robustness by up to 3%, 2%, and 3% under FGSM, PGD and BIM attacks, respectively.

5.4.2. Train with adversarial examples

Then, we validate the effectiveness of our method by comparing it with adversarial training-based approaches, (*i.e.*, AT [27], RAT [8], and SR [25]). Table 1 presents the classification accuracy of the compared methods under various attack scenarios. It can be observed that, our proposed TG-Reg consistently outperforms other SOTA methods. For instance, on VGG11 with CIFAR10 at $T = 4$, under BIM attack, TG-Reg achieves 9% increase in classification accuracy against RAT. Compared to the SR strategy, TG-Reg delivers a substantial improvement in adversarial robustness, achieving a 10-percentage-point gain across all evaluated attacks.

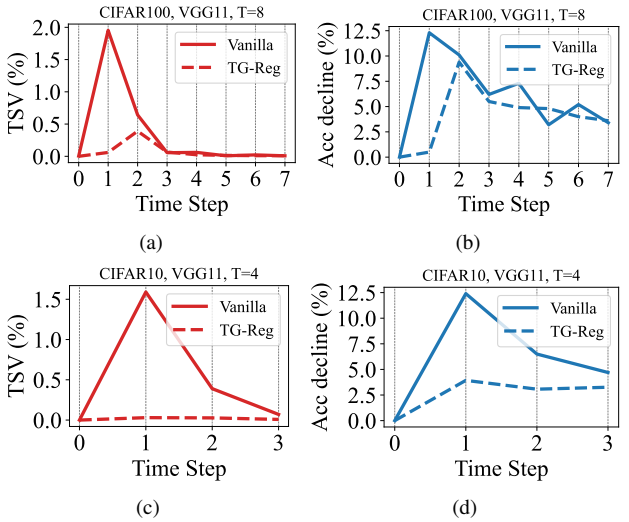


Figure 3. The changing of TSV ((a) and (c)) and accuracy decline ((b) and (d)) at each time step before and after TG-Reg.

5.4.3. Under black-box attacks

Table 2 shows the performance of our proposed TG-Reg on black box attack. We use the same model trained with a different seed to generate perturbed images. In black box setting, for the models on both datasets, we can also observe the superior performance of our TG-Reg. In particular, on VGG11 with CIFAR100, TG-Reg achieves 35.91% accuracy attacked by PGD, while vanilla is 5.77%.

5.4.4. Effectiveness of TG-Reg through TSV

As analyzed above, the robustness of SNNs at each time step is constrained by the proposed temporal sensitivity value (TSV), as further illustrated in Figure 2. In this section, we demonstrate that the proposed temporal granularity regularization (TG-Reg) improves robustness by reducing TSV on the temporal characteristics of SNNs. Figure 3 presents the TSV ((a) and (c)) and the accuracy decline ((b) and (d)) at each time step of different models, both before and after applying TG-Reg. As shown, embedding TG-Reg into SNNs results in lower TSV at each time step (indicated by the dotted red lines in Figure 3(a) and (c)) compared to the original model without TG-Reg. Similarly, the accuracy decline (as depicted by the dotted blue lines in Figure 3(b)

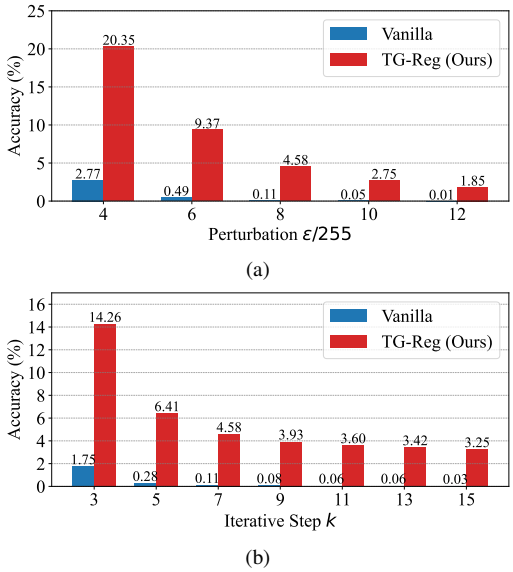


Figure 4. Performance of TG-Reg and Vanilla SNN under PGD with different perturbation ϵ (a) and different iterative step k (b).

and (d)) is also reduced after TG-Reg. These experimental results confirm the effectiveness of TG-Reg in enhancing the model's robustness, aligning well with the proposed TSV analysis.

5.4.5. Performance under Different perturbation ϵ and iterative step k

We present the accuracy of vanilla and TG-Reg under PGD attacks with varying perturbation ϵ, and iterative steps k, in Figures 4(a) and 4(b), respectively. The results demonstrate that TG-Reg models exhibit a slower decline in accuracy compared to vanilla models. As shown in Figure 4(a), the classification accuracy of the vanilla SNN decreases with increasing ϵ and eventually drops to 0%, while TG-Reg models maintain higher accuracy under larger perturbations. Furthermore, we evaluate robustness against stronger attacks by increasing the number of iterative steps k, as depicted in Figure 4(b). Although robustness of both models decreases with increasing k, TG-Reg models consistently outperform the vanilla models. These results confirm that TG-Reg enhances the robustness of SNNs effectively and reliably without relying on gradient obfuscation [1].

5.4.6. Computational Costs

Attacking time. The attacking time of the proposed TG-attack alongside other methods is shown in Table 3. As observed, TG-attack exhibits the same attack efficiency with other methods.

Training time. The training time of the proposed method alongside other methods is shown in Table 4. As observed, it is evident that TG-Reg do increase the training time of vanilla method, this is due to the additional temporal granularity gradient calculation in the regularization term. In future work, we will also strive to further optimize the calculation of gradients on time steps to reduce computational overhead. However, TG-Reg exhibits better computational efficiency than adversarial training-based methods (*i.e.*, AT and RAT), which achieves a favorable balance between efficiency and robustness.

Table 3. Attacking time of TG-Attack. The dataset is CIFAR10 with $T = 4$, the network is ResNet34.

Methods	Attacking time
FGSM	$34s$
TG-FGSM (Ours)	$33s$
PGD	$199s$
TG-PGD (Ours)	$194s$
BIM	$204s$
TG-BIM (Ours)	$204s$

Table 4. Training time of TG-Reg. The dataset is CIFAR100 with $T = 8$, the network is VGG11. The results are computed for the mean of 200 epochs for the dataset, "s" represents the second.

Methods	Training time
Vanilla	$108s \pm 7s$
AT	$300s \pm 6s$
RAT	$294s \pm 4s$
TG-Reg (Ours)	$236s \pm 6s$

6. Conclusion

Unlike previous work that investigates the robustness of SNNs from the perspective of overall temporal characteristics, which based on *average gradient* for adversarial test, this paper introduces a novel approach by incorporating the concept of temporal granularity (*i.e.*, examining at individual time steps). Specifically, we first propose a temporal granularity attack (TG-attack) that leverages the *gradient at each time step* of SNNs. Our TG-attack not only captures the robustness of SNNs across different time steps but also shows superior attack efficacy compared to traditional methods. Building on experimental observations, we theoretically derive that the robustness of SNNs at a specific time step is governed by the Hessian matrix of the input-output gradient at that time step, which we define as Temporal Sensitivity (TS). By calculating the Temporal Sensitivity Values (TSV) at various time steps, we can assess the robustness of SNNs without the need to generate adversarial examples explicitly. Finally, we introduce a temporal granularity robustness regularization term (TG-Reg) that constrains the Temporal Sensitivity Values (TSV) at each time step, thereby effectively enhancing the overall robustness of SNNs. Experimental results show that our approach outperforms existing state-of-the-art methods.

7. Acknowledgements

This work was supported in part by the STI 2030 Major Projects under Grant 2021ZD0200400, in part by the National Natural Science Foundation of China (62376247, 62436008, 62334014), in part by the grants from Key R&D Program of Zhejiang (2022C01048), in part by Fundamental Research Funds for the Central Universities (226-2025-00057), and in part by Chongqing Natural Science Foundation Innovation and Development Joint Fund (No. CSTB2025NSCQ-LZX0160).

References

[1] Anish Athalye, Nicholas Carlini, and David Wagner. Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples. In *Proc. of International Conference on Machine Learning*, pages 274–283. PMLR, 2018. 5, 8

[2] Anish Athalye, Logan Engstrom, Andrew Ilyas, and Kevin Kwok. Synthesizing robust adversarial examples. In *Proc. of International Conference on Machine Learning*, pages 284–293. PMLR, 2018. 5

[3] Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *IEEE symposium on security and privacy (sp)*, pages 39–57. IEEE, 2017. 5

[4] Hojae Choi, Jaewook Kim, Jongkil Park, Seongsik Park, Hyun Jae Jang, Seung Hwan Lee, Byeong-Kwon Ju, and YeonJoo Jeong. STAR-SNN: A spatio-temporal adaptive recurrent spiking neural network with separated propagation surrogate gradient for hardware efficient real-time learning. *Neurocomputing*, page 132968, 2026. 1

[5] Francesco Croce and Matthias Hein. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. In *Proc. of International Conference on Machine Learning*, pages 2206–2216. PMLR, 2020. 5

[6] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *Proc. of Computer Vision and Pattern Recognition*, pages 248–255. IEEE, 2009. 5, 6

[7] Shikuang Deng, Yuhang Li, Shanghang Zhang, and Shi Gu. Temporal efficient training of spiking neural network via gradient re-weighting. In *International Conference on Learning Representations*, 2022. 1, 3, 5

[8] Jianhao Ding, Tong Bu, Zhaofei Yu, Tiejun Huang, and Jian Liu. SNN-RAT: Robustness-enhanced spiking neural network through regularized adversarial training. *Advances in Neural Information Processing Systems*, 35:24780–24793, 2022. 1, 2, 6, 7

[9] Jianhao Ding, Zhaofei Yu, Tiejun Huang, and Jian K Liu. Spike timing reshapes robustness against attacks in spiking neural networks. *arXiv preprint arXiv:2306.05654*, 2023. 1, 3

[10] Jianhao Ding, Zhiyu Pan, Yujia Liu, Zhaofei Yu, and Tiejun Huang. Robust stable spiking neural networks. In *Proc. of International Conference on Machine Learning*, pages 11016–11029. PMLR, 2024. 6, 7

[11] Jianhao Ding, Zhaofei Yu, Tiejun Huang, and Jian K Liu. Enhancing the robustness of spiking neural networks with stochastic gating mechanisms. In *AAAI Conference on Artificial Intelligence*, pages 492–502, 2024. 1, 2, 6, 7

[12] Rida El-Allami, Alberto Marchisio, Muhammad Shafique, and Ihsen Alouani. Securing deep spiking neural networks against adversarial attacks through inherent structural parameters. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 774–779. IEEE, 2021. 1, 2

[13] Mehrshad Golesorkhi, Javier Gomez-Pilar, Federico Zilio, Nareg Berberian, Annemarie Wolff, Mustapha CE Yagoub, and Georg Northoff. The brain and its time: intrinsic neural timescales are key for input processing. *Communications biology*, 4(1):970, 2021. 1

[14] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014. 3, 5

[15] Yufei Guo, Xiaode Liu, Yuanpei Chen, Weihang Peng, Yuhan Zhang, and Zhe Ma. Spiking transformer: Introducing accurate addition-only spiking self-attention for transformer. In *Proc. of Computer Vision and Pattern Recognition*, pages 24398–24408. IEEE, 2025. 1

[16] John R Hershey and Peder A Olsen. Approximating the kullback leibler divergence between gaussian mixture models. In *International Conference on Acoustics, Speech and Signal Processing*, pages IV–317. IEEE, 2007. 4

[17] Youngeun Kim, Hyoungseob Park, Abhishek Moitra, Abhiroop Bhattacharjee, Yeshwanth Venkatesha, and Priyadarshini Panda. Rate coding or direct coding: Which one is better for accurate, robust, and energy-efficient spiking neural networks? In *International Conference on Acoustics, Speech and Signal Processing*, pages 71–75. IEEE, 2022. 3

[18] Youngeun Kim, Yuhang Li, Hyoungseob Park, Yeshwanth Venkatesha, Anna Hambitzer, and Priyadarshini Panda. Exploring temporal information dynamics in spiking neural networks. In *AAAI Conference on Artificial Intelligence*, pages 8308–8316, 2023. 1, 4

[19] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009. 5, 6

[20] Souvik Kundu, Massoud Pedram, and Peter A Beerel. HIRE-SNN: Harnessing the inherent robustness of energy-efficient deep spiking neural networks by training with crafted input noise. In *Proc. of International Conference on Computer Vision*, pages 5209–5218, 2021. 1, 2

[21] Alexey Kurakin, Ian J Goodfellow, and Samy Bengio. Adversarial examples in the physical world. In *Artificial intelligence safety and security*, pages 99–112. Chapman and Hall/CRC, 2018. 3, 5

[22] Hongmin Li, Hanchao Liu, Xiangyang Ji, Guoqi Li, and Luping Shi. Cifar10-DVS: an event-stream dataset for object classification. *Frontiers in neuroscience*, 11:244131, 2017. 5, 6

[23] Yuhang Li, Tamar Geller, Youngeun Kim, and Priyadarshini Panda. Seenn: Towards temporal spiking early exit neural networks. *Advances in Neural Information Processing Systems*, 36, 2024. 1

[24] Seppo Linnainmaa. Taylor expansion of the accumulated rounding error. *BIT Numerical Mathematics*, 16(2):146–160, 1976. 4

[25] Yujia Liu, Tong Bu, Jianhao Ding, Zecheng Hao, Tiejun Huang, and Zhaofei Yu. Enhancing adversarial robustness in snns with sparse gradients. In *Proc. of International Conference on Machine Learning*, pages 30738–30754. PMLR, 2024. 3, 5, 6, 7

[26] De Ma, Xiaofei Jin, Shichun Sun, Yitao Li, Xundong Wu, Youneng Hu, Fangchao Yang, Huajin Tang, Xiaolei Zhu, Peng Lin, et al. Darwin3: a large-scale neuromorphic chip with a novel isa and on-chip learning. *National Science Review*, 11(5):nwae102, 2024. 1

[27] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations*, 2018. 3, 5, 6, 7

[28] Alberto Marchisio, Giorgio Nanfa, Faiq Khalid, Muhammad Abdullah Hanif, Maurizio Martina, and Muhammad Shafique. Is spiking secure? a comparative study on the security vulnerabilities of spiking and deep neural networks. In *2020 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2020. 2

[29] Takeru Miyato, Shin-ichi Maeda, Masanori Koyama, and Shin Ishii. Virtual adversarial training: a regularization method for supervised and semi-supervised learning. *IEEE transactions on pattern analysis and machine intelligence*, 41(8):1979–1993, 2018. 4

[30] Bhaskar Mukhoty, Hilal AlQuabeh, and Bin Gu. Improving generalization and robustness in snns through signed rate encoding and sparse encoding attacks. In *The Thirteenth International Conference on Learning Representations*, 2025. 1

[31] Saima Sharmin, Priyadarshini Panda, Syed Shakib Sarwar, Chankyu Lee, Wachirawit Ponghiran, and Kaushik Roy. A comprehensive analysis on adversarial robustness of spiking neural networks. In *2019 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2019. 2

[32] Saima Sharmin, Nitin Rathi, Priyadarshini Panda, and Kaushik Roy. Inherent adversarial robustness of deep spiking neural networks: Effects of discrete input encoding and nonlinear activations. In *European Conference on Computer Vision*, pages 399–414. Springer, 2020. 1, 2

[33] Hangchi Shen, Qian Zheng, Huamin Wang, and Gang Pan. Rethinking the membrane dynamics and optimization objectives of spiking neural networks. *Advances in Neural Information Processing Systems*, 37:92697–92720, 2024. 1

[34] Doron Tal and Eric L Schwartz. Computing with the leaky integrate-and-fire neuron: logarithmic computation and multiplication. *Neural computation*, 9(2):305–318, 1997. 3

[35] Cheng Wang, Chankyu Lee, and Kaushik Roy. Noise resilient leaky integrate-and-fire neurons based on multi-domain spintronic devices. *Scientific Reports*, 12(1):8361, 2022. 1, 2, 3

[36] Keming Wu, Man Yao, Yuhong Chou, Xuerui Qiu, Rui Yang, Bo Xu, and Guoqi Li. RSC-SNN: Exploring the trade-off between adversarial robustness and accuracy in spiking neural networks via randomized smoothing coding. In *Proc. of ACM Multimedia*, pages 2748–2756, 2024. 1, 3

[37] Yujie Wu, Lei Deng, Guoqi Li, and Luping Shi. Spatio-temporal backpropagation for training high-performance spiking neural networks. *Frontiers in neuroscience*, 12: 323875, 2018. 1

[38] Yujie Wu, Lei Deng, Guoqi Li, Jun Zhu, Yuan Xie, and Luping Shi. Direct training for spiking neural networks: Faster, larger, better. In *AAAI Conference on Artificial Intelligence*, pages 1311–1318, 2019. 1

[39] Weizhen Xie, John H Wittig Jr, Julio I Chapeton, Mostafa El-Kalliny, Samantha N Jackson, Sara K Inati, and Kareem A Zaghloul. Neuronal sequences in population bursts encode information in human cortex. *Nature*, pages 1–8, 2024. 1

[40] Mengting Xu, De Ma, Huajin Tang, Qian Zheng, and Gang Pan. FEEL-SNN: Robust spiking neural networks with frequency encoding and evolutionary leak factor. *Advances in Neural Information Processing Systems*, 37:91930–91950, 2024. 1, 3, 6, 7

[41] Jiaqi Yan, Changping Wang, De Ma, Huajin Tang, Qian Zheng, and Gang Pan. Training high performance spiking neural network by temporal model calibration. In *Proc. of International Conference on Machine Learning*. PMLR, 2025. 1

[42] Xingyu Zhang, Yu Cao, Jian Huang, Jindong Liu, and Zhi-Qiang Zhang. A systematic review of spiking neural networks for human-robot interaction in rehabilitative wearable robotics. *IEEE Transactions on Cognitive and Developmental Systems*, 2025. 1